

AI翻訳チャット+通話アプリ（仮）

開発設計統合報告書 (Step 1 - Step 4)

役割：	チーフフロントエンドアーキテクト 兼 UI/UX品質監査責任者
開発環境：	VSCode / Live Server環境 / 各自のローカルポート
主要技術：	HTML5 / Tailwind CSS (CDN版) / Vanilla JavaScript

1. 開発基本方針 & アーキテクチャ概要

本プロジェクトは、18日までの極めてタイトなスケジュールと、チーム内（JS担当・Python担当）での結合の手軽さを最優先考慮し、「**HTML/CSS/JS一体型モックアップモデル**」を標準として採用します。外部の重厚なビルド環境やサーバー設定なしで、VSCodeのLive Server等を通じてブラウザ単体で完全に動作するプロトタイプを構築します。

- **主要UIフレームワーク：** Tailwind CSS (CDN版) を利用し、すべてのスタイリングをHTML上のユーティリティクラス（flex, grid, bg-white 等）で完結させます。
- **命名規則：** **** DOM操作およびバックエンド連携の変数衝突を防ぐため、HTMLのID名は**ケバブケース**（send-btn, chat-input）、JavaScriptの関数・変数名は****キャメルケース****（sendMessage, isTranslating）で完全統一します。

2. 画面・UIコンポーネント設計

① 画面構成・レイアウト配置

エリア	主な役割・Tailwind CSS配置方針
全体コンテナ	max-w-md mx-auto h-screen flex flex-col bg-slate-50 relative スマートフォン表示に最適な縦長画面をデスクトップ中央に擬似固定。
ヘッダー（固定）	flex justify-between items-center p-4 bg-white border-b border-slate-200 左側に相手情報（アバター・国旗）、右側に通話開始（call-btn）を配置。
中央チャットエリア	flex-1 overflow-y-auto p-4 space-y-4 履歴増大時も画面を突き破らず、エリア内だけで縦スクロールを完結。
通話中オーバーレイ	absolute inset-0 hidden bg-slate-900/95 z-50 flex flex-col 初期状態は非表示（hidden）。通話開始時にチャット前面を完全カバー。
下部入力フッター	p-4 bg-white border-t border-slate-200 flex items-center gap-2 画面最下部に固定。翻訳トグル、テキスト入力、送信ボタンを横並び配置。

② 主要UIコンポーネント仕様定義

・チャットメッセージ吹き出し（自分 / 相手）：

クラス定義に `max-w-[75%] break-all whitespace-pre-wrap rounded-2xl p-3` を強制します。海外ユーザーがスペースのない極端な英単語の長文や、長いURLを入力した際、吹き出しが画面外へ突き抜けてレイアウトが崩壊（UI崩壊）するのを完全に防御します。

・翻訳切り替えスイッチ（translate-switch）：

視覚的にON/OFF（緑とグレー）が即座に判別できるトグルスイッチ型コンポーネント。AI翻訳の有効・無効状態をユーザーが直感的に認識できるようにします。

3. 画面遷移 & UI制御フロー

① 制御フローの定義

- ・メッセージ送信時：`** chat-input`の値から空白を除去（`trim()`）。空文字（`""`）であれば処理を中断し、空の吹き出しが生成されるバグを防ぎます。追加後は自動で最下部までスクロール（`scrollTop = scrollHeight`）させます。
- ・通話モック切り替え：`** call-btn`押下で`call-overlay`の`hidden`を解除。`setInterval`で疑似タイマーを起動。`hangup-btn`押下でタイマーをクリア（`clearInterval`）し、再度`hidden`を付与してチャット画面へ戻します。

4. ディレクトリ・ファイル構成（VSCode対応）

```
my-chat-app/  
├── index.html      # 必須画面（チャット・通話・設定）と全JSロジックを1ファイルに集約  
└── README.md      # 開発メンバー（JS担当・Python担当）への引き継ぎ・仕様メモ
```

5. UI品質・フロントエンドセキュリティ仕様（監査クリア要件）

【重要】フロントエンド堅牢性ポリシー（XSS対策）

ユーザー入力メッセージ、および将来的にAIから返却される翻訳テキストを画面にレンダリングする際は、絶対に `innerHTML` を使用せず、`textContent` を徹底します。これにより、悪意あるスクリプトコードが注入されてもブラウザ上では単なる「文字列」として無害化され、実行を完全に防御します。

- 多重・空送信ガード：送信ボタンが押された瞬間、一時的に `disabled = true` を付与し、DOM描画が安全に完了した後に解除する制御を入れ、連打によるバグを防ぎます。
- バックエンドエラー表示の割り込み設計：将来的にPython側の翻訳通信などでエラーが発生した際、画面が真っ白にフリーズするのを防ぐため、送信したメッセージのすぐ下に ⚡ 翻訳に失敗しました。再試行してください。という赤字（`text-red-500 text-xs`）の警告UIを動的にインサートできる設計をあらかじめ組み込みます。